



CẨM NANG TOÀN DIỆN THIẾT LẬP VÀ VẬN HÀNH HỆ THỐNG QA AGENT CHUYÊN SÂU

Kiến Trúc Zero-CLI, Knowledge-First & Multi-Agent Testing Pipeline

CO-WELL Tech Academy

Đội ngũ Nghiên cứu & Phát triển Giải pháp AI Kiểm Thử

Hà Nội, Tháng 09/2026

MỤC LỤC TỔNG QUAN

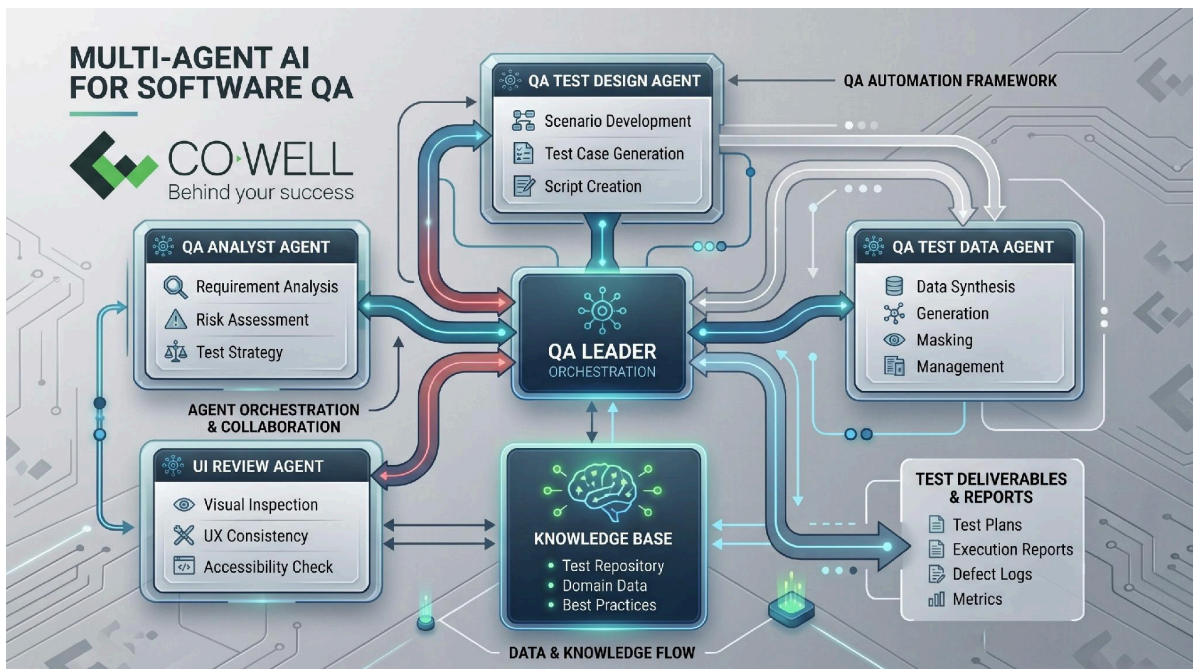
Chương 1	Giới Thiệu Chung & 4 Trụ Cột Triết Lý Hệ Thống	03
Chương 2	Khởi Tạo Môi Trường & Phân Định Phân Vùng Làm Việc	06
Chương 3	Cổng Tiếp Nhận Số 0 (Gate 0) & Bộ Chuyển Đổi Đa Định Dạng	09
Chương 4	Bộ Não Tri Thức Vĩnh Viễn (SSOT Knowledge)	12
Chương 5	Mô Hình Điều Phối Của QA Leader	16
Chương 6	Quy Trình Thiết Kế Kiểm Thử Chuẩn 6 Chặng	19
Chương 7	Xử Lý Quy Mô Lớn & Động Cơ Sinh Dữ Liệu Tự Động	23
Chương 8	Kịch Bản Vận Hành 100% Ngôn Ngữ Tự Nhiên (Zero-CLI)	27
Chương 9	Tích Hợp Hệ Thống Quản Lý Kiểm Thử (Jira / Redmine)	31
Chương 10	Quản Trị Hệ Thống & Bác Sĩ Agent (Agent Doctor)	35
Chương 11	Cẩm Nang Thực Chiến: Hướng Dẫn Hội Thoại & Kịch Bản Gỡ Chat Thực Tế	39

CHƯƠNG 1

GIỚI THIỆU CHUNG & 4 TRỤ CỘT TRIẾT LÝ HỆ THỐNG

1.1. Tầm Nhìn & Mục Tiêu Cốt Lõi

Hệ thống CW QA Agent Ecosystem là bộ giải pháp toàn diện ứng dụng trí tuệ nhân tạo tác tử (Agentic AI) vào quy trình đảm bảo chất lượng phần mềm (Software Quality Assurance). Hệ thống không thay thế con người mà hoạt động như một đội ngũ chuyên gia QA ảo gồm 8 chuyên khoa độc lập, song hành cùng Tester, Business Analyst (BA), Product Owner (PO) và Developer để loại bỏ sai sót logic ngay từ giai đoạn tài liệu yêu cầu cho tới khi kiểm thử tự động trên môi trường thực tế.



Hình 1.1: Sơ đồ tổng quan kiến trúc hệ sinh thái 8 chuyên gia QA Agent & luồng tương tác Một Cửa

1.2. Bốn Trụ Cột Triết Lý Kỹ Thuật Bất Biến

Hệ thống được thiết kế dựa trên 4 nguyên tắc kỹ thuật cốt tử nhằm đảm bảo độ chính xác tuyệt đối, triệt tiêu ảo giác (hallucination) và tối ưu hóa tài nguyên token:

- **1. Mô hình Một cửa qua QA Leader (Single Point of Contact):** Người dùng (Tester, BA, PO, Quản lý) không cần nhớ danh tính hay trực tiếp tương tác với các sub-agent con. Mọi giao tiếp diễn ra duy nhất qua QA Leader (agents/qa-lead/). QA Leader tự động nắm bắt ý định, tra cứu bản đồ hệ thống, lập kế hoạch và điều phối chuyên gia tương ứng.
- **2. Trải nghiệm 100% Bằng Lời Nói Tự Nhiên (Zero-CLI):** Người dùng tuyệt đối không cần mở terminal để gõ bất kỳ lệnh script hay npm run... Toàn bộ các công cụ (intake, convert, map:sync, data:gen, jira:push...) là công cụ nội bộ chạy ngầm ở hậu trường do AI tự động kích hoạt.
- **3. Tri Thức Là Cốt Lõi Vĩnh Viễn (SSOT – Single Source of Truth):** Toàn bộ tri thức nghiệp vụ, quy tắc được xác nhận và câu trả lời của BA được tích lũy vào thư mục knowledge/. Lần chạy sau kế thừa lần chạy trước; hệ thống tuyệt đối không bao giờ hỏi lại những điều con người đã giải quyết.
- **4. Chống Tràn Token (Plan-First & Blueprint Batching):** Trước khi thực thi bất kỳ tính năng nào, Agent bắt buộc phải khởi tạo kế hoạch chia nhỏ milestone tại OUTPUT/<slug>/OO_plan.md. Khi sinh số lượng lớn test cases (từ 100 đến 1.000 test cases), hệ thống áp dụng kỹ thuật sinh bản thiết kế khung (Blueprint JSON) và chia lô (Chunked Batching) để tránh cắt cụt context.

Trụ cột triết lý	Giá trị thực tiễn mang lại cho dự án
Zero-CLI	Dân chủ hóa AI cho toàn bộ đội ngũ Tester, BA, Quản lý không chuyên kỹ thuật dòng lệnh.
System Map First	Cấm Agent đọc quét dò mò file; mở đúng file đích giúp tiết kiệm 80% lượng token tiêu thụ.
Plan-First	Tránh quá tải context window, cho phép chuyển giao mạch lạc giữa các công cụ AI khác nhau.
Chuẩn FACT 100%	100% dữ kiện phải dẫn xuất từ tài liệu nguồn hoặc tri thức; cấm tự bịa giả định.

Bảng 1.1: Tóm tắt giá trị của 4 trụ cột thiết kế

1.3. Danh Mục 8 Chuyên Khoa QA Agent Nội Bộ

Tên Agent	Đường dẫn danh tính	Phạm vi chuyên môn duy nhất
qa-lead	agents/qa-lead/	Tổng chỉ huy điều phối, tiếp nhận Gate 0, lập plan và nghiệm thu.
qa-analyst	agents/qa-analyst/	Đọc hiểu yêu cầu, phân tích rủi ro, quét kẻ hở O6W, thiết kế test idea.
qa-test-design	agents/qa-test-design/	Thiết kế test case 8 trường, chia lô blueprint, rà soát độ phủ 3 góc nhìn.
qa-test-data	agents/qa-test-data/	Thiết kế Data Class, sinh dữ liệu tự động (0 LLM token), ma trận biên.
qa-ui-review	agents/qa-ui-review/	Phân tích ảnh chụp màn hình UI (Vision AI), màu sắc, font, WCAG.
qa-exploratory	agents/qa-exploratory/	Kiểm thử thăm dò theo charter (SBTM), khám phá hành trình web thực tế.
qa-reporter	agents/qa-reporter/	Chuẩn hóa bug report 7 trường và lập báo cáo tổng hợp QA hàng ngày.
qa-automation	agents/qa-automation/	Gom cụm luồng hành trình, sinh Page Object Model và chạy test Playwright E2E.

Bảng 1.2: Phân định trách nhiệm của 8 nhóm chuyên gia QA

CHƯƠNG 2: KHỞI TẠO MÔI TRƯỜNG & PHÂN ĐỊNH PHÂN VÙNG

2.1. Bản Đồ Không Gian Làm Việc (Workspace Boundaries)

Hệ thống phân định nghiêm ngặt không gian làm việc thành 4 khu vực chức năng biệt lập. Mọi AI Agent chỉ được phép đọc và ghi đúng phân vùng quy định, tuyệt đối không được làm ô nhiễm thư mục gốc:

⚡ BẢN ĐỒ 4 PHÂN VÙNG CHỨC NĂNG BẤT BIẾN

[PHÂN VÙNG 1] INPUT/<task-slug>/ (CHỈ ĐỌC): Chứa tài liệu thô từ 5 đối tác: 01_business, 02_ba (PRD/SRS), 03_dev (API/DB), 04_design (UI), 05_communication (Q&A). Cấm Agent chỉnh sửa tài liệu nguồn.

[PHÂN VÙNG 2] OUTPUT/<task-slug>/ (CHỈ GHI): Nơi DUY NHẤT được phép xuất file. Lưu trữ Master Spec (00_plan đến 06_coverage_review) và Tầng thực thi độc lập (runs/RUN-XX-<ticket>/).

[PHÂN VÙNG 3] knowledge/ (SSOT VĨNH VIỄN): Bộ não tri thức của dự án. Gồm _system_map.json (định tuyến vệ tinh), _project.md, _glossary.md, và features/<slug>.md (tri thức nghiệp vụ tích lũy).

[PHÂN VÙNG 4] agents/ (HỆ THỐNG CHUYÊN GIA): Chứa qa-lead/ (tổng chỉ huy một cửa), 7 sub-agent chuyên trách và thư mục tools/ chứa 12 script nội bộ chạy ngầm.

Hình 2.1: Sơ đồ phân định ranh giới không gian làm việc và quyền truy cập của Agent

Khu vực	Chức năng duy nhất	Quy tắc truy cập của AI Agent
INPUT/<slug>/	Lưu trữ tài liệu đầu vào từ 5 đối tác: kinh doanh, BA, kỹ thuật, UI/UX, trao đổi.	CHỈ ĐỌC. Cấm Agent ghi đè hoặc chỉnh sửa tài liệu nguồn thô của đối tác.
OUTPUT/<slug>/	Lưu trữ Master Spec (00 đến 06) và Tầng thực thi đợt chạy (runs/).	NƠI DUY NHẤT ĐƯỢC PHÉP XUẤT FILE. Tuyệt đối không sinh file rác ở thư mục gốc.
knowledge/	Bộ não tri thức vĩnh viễn của dự án (SSOT).	BẮT BUỘC ĐỌC ĐẦU TIÊN. Agent đọc _system_map.json để định tuyến file đích.
agents/	8 nhóm chuyên gia QA và công cụ thực thi nội bộ.	Chỉ giao tiếp qua QA Leader, không gọi rời rạc các sub-agent.

Bảng 2.1: Nguyên tắc vận hành 4 phân vùng chức năng

2.2. Quy Trình Khởi Tạo Dự Án Mới (Self-Bootstrap Knowledge)

Khi bắt đầu một dự án mới hoàn toàn chưa có tri thức nền tảng:

Bước 1: Cài đặt gói phụ thuộc. Cài đặt các thư viện chuyển đổi văn bản và tiện ích nội bộ:

```
[Terminal (bash)]
npm install
```

Bước 2: Tự động khởi tạo tri thức hạt giống (Seed Templates). Sao chép bộ hạt giống sạch từ `agents/templates/knowledge/` sang `knowledge/`:

```
[Bootstrap Command (bash)]
npm run knowledge:init
```

Hệ thống sẽ tự động sinh các file nền tảng sạch bao gồm:

- **knowledge/_system_map.json:** Bản đồ hệ thống với `features_inventory` sẵn sàng đón nhận tính năng mới.
- **knowledge/_project.md:** Template quy ước dùng chung (tiền tệ, định dạng ngày tháng, timezone, quy chuẩn định danh mã).
- **knowledge/_glossary.md:** Template từ điển thuật ngữ nghiệp vụ thống nhất giữa BA, Dev và QA.
- **knowledge/_template.md:** Mẫu chuẩn 8 phần bắt buộc để tạo tri thức cho từng tính năng mới.

Bước 3: Kiểm tra sức khỏe toàn vẹn hệ sinh thái (Agent Doctor). Trước khi đưa vào vận hành thực tế, kích hoạt chẩn đoán toàn vẹn hệ thống:

```
[Health Check (bash)]
npm run agent:check
```

Kết quả chẩn đoán cần hiển thị 100% OK cho toàn bộ 8 nhóm chuyên gia và 12 công cụ nội bộ.

2.3. Cấu Hình Môi Trường Tích Hợp (.env)

Để kết nối tự động với Jira Cloud, Jira Server hoặc Redmine, tạo file `.env` tại thư mục gốc:

```
[Cấu hình môi trường .env (env)]
# Cấu hình Jira Cloud / Jira Xray
JIRA_HOST=https://your-domain.atlassian.net
JIRA_EMAIL=qa-lead@example.com
JIRA_API_TOKEN=your_jira_api_token
JIRA_PROJECT_KEY=PROJECT_KEY

# Cấu hình Redmine (Tùy chọn)
REDMINE_HOST=https://redmine.your-domain.com
REDMINE_API_KEY=your_redmine_api_key
```



CHẾ ĐỘ XUẤT TẬP THỦ CÔNG

Nếu chưa cấu hình file `.env`, hệ thống vẫn hoạt động bình thường ở chế độ xuất file CSV chuẩn UTF-8 BOM để người dùng import thủ công trực tiếp vào Jira Xray và Redmine mà không cần API token.

CHƯƠNG 3: CÔNG TIẾP NHẬN & BỘ CHUYỂN ĐỔI ĐA ĐỊNH DẠNG

3.1. Cổng Tiếp Nhận Số 0 Của QA Leader (Intake Gate)

Mọi dự án phần mềm thực tế đều bắt đầu từ việc tiếp nhận nhiều loại tài liệu rời rạc từ các đối tác liên quan. QA Leader đóng vai trò là người gác cổng số 0, chịu trách nhiệm chuẩn hóa, phân loại thông minh tài liệu vào 5 ngăn đối tác và căn chỉnh mục tiêu kiểm thử trước khi phân rã cho các chuyên gia phân tích:

⚡ QUY TRÌNH TIẾP NHẬN, CHUYỂN ĐỔI VÀ PHÂN LOẠI CỔNG SỐ 0

[BƯỚC 1] Tiếp Nhận Tài Liệu Thô: Quét các tệp nguồn từ đối tác (.docx, .xlsx, .pdf, OpenAPI .json, Postman .yaml, .txt).



[BƯỚC 2] Universal Doc Converter: Kích hoạt intake.js tự động chuyển đổi sang Markdown sạch, unescape ký tự, trích xuất bảng biểu.



[BƯỚC 3] Phân Loại 5 Ngăn Đối Tác: Tự động đưa tài liệu vào đúng ngăn: 01_business, 02_ba, 03_dev, 04_design, 05_communication.



[BƯỚC 4] Gap Assessment (Kiểm Tra Lỗ Hổng): Bắt buộc kiểm tra có tài liệu trong 02_ba/ hay không. Nếu thiếu 02_ba/ -> DỪNG LẠI (Verdict: ASK).



[BƯỚC 5] Outcome Alignment (Căn Chỉnh Mục Tiêu): Xác định rõ kỳ vọng của người dùng theo 4 Mode làm việc và khởi tạo OUTPUT/<slug>/OO_plan.md.

Hình 3.1: Sơ đồ luồng xử lý tự động tại Cổng tiếp nhận số 0 của QA Leader

3.2. Cơ Cấu 5 Ngăn Đối Tác Tại INPUT/<task-slug>/

- **1. 01_business/:** Định hướng kinh doanh, bài toán thương mại, chính sách cấp cao, KPI sản phẩm.
- **2. 02_ba/ [BẮT BUỘC]:** PRD (Product Requirement Document), SRS, User Stories, Use Cases, bảng Acceptance Criteria. Thiếu ngăn này thì pipeline kiểm thử không được phép kích hoạt.
- **3. 03_dev/:** Tài liệu kỹ thuật, API Swagger/OpenAPI spec, Postman collection, sơ đồ cơ sở dữ liệu (DB Schema), DDL script.
- **4. 04_design/:** Liên kết Figma, UI wireframes, ảnh chụp màn hình thiết kế giao diện (UI mockups).
- **5. 05_communication/:** Biên bản cuộc họp làm rõ yêu cầu, Q&A log giữa QA và khách hàng, Change Requests (CR).

3.3. Đánh Giá Khoảng Trống (Gap Assessment) & Căn Chỉnh Mục Tiêu (Outcome Alignment)

Trước khi bắt tay vào thiết kế, QA Leader thực hiện 2 kiểm định nền tảng:

- **Đánh giá sự thiếu hụt tài liệu (Gap Assessment):** Kiểm tra bắt buộc phải có tài liệu trong 02_ba/. Nếu dự án thiếu các ngăn phụ trợ (03_dev, 04_design...), QA Leader sẽ không dừng lại mà ghi nhận các khoảng trống đổ vào mục rủi ro của OO_plan.md và chủ động gắn nhãn [GIẢ ĐỊNH] cho các chi tiết kỹ thuật liên quan.
- **Căn chỉnh kết quả đầu ra (Outcome Alignment):** Xác định rõ kỳ vọng của người dùng theo 4 Mode làm việc cụ thể:

- **Mode 1 – Manual Test Cases Only:** Chỉ phân tích nghiệp vụ, quét kế hoạch O6W và xuất đặc tả Test Case 8 trường (01 đến 06).
- **Mode 2 – Manual + Test Data:** Bổ sung thêm thiết kế Data Class, sinh dữ liệu tự động (0 LLM Token) và ma trận truy vết dữ liệu (09 đến 12).
- **Mode 3 – Web Journey & Gherkin:** Khám phá giao diện web thực tế và chuẩn hóa kịch bản hành trình theo chuẩn Gherkin BDD (Given - When - Then).
- **Mode 4 – Full Automation E2E:** Gom cụm luồng, tự động sinh Page Object Model (POM) và chạy kịch bản Playwright E2E bắt video/ảnh bằng chứng thực thi.

3.4. Bộ Chuyển Đổi Đa Định Dạng & Xử Lý Hậu Kỳ (doc-converter.js)

Định dạng nguồn	Thư viện xử lý	Quy trình làm sạch & Tối ưu token
Word (.docx)	mammoth	Kích hoạt bộ lọc cleanMarkdown(): unescape triệt để ký tự \(\, \), \., \-, _;; xóa thẻ rác HTML ; chuyển bảng HTML sang cú pháp Markdown chuẩn.
Excel (.xlsx, .xls, .csv)	xlsx (SheetJS)	Duyệt toàn bộ các sheet, tự động trích xuất bảng dữ liệu ma trận (RTM, danh mục lỗi, rules), format ngày tháng và escape ký tự .
PDF (.pdf)	pdf-parse	Trích xuất toàn bộ văn bản nhiều trang thuần Node.js (không cần môi trường Python phụ thuộc), tự động phân cách trang bằng ---.
OpenAPI / Swagger (.json, .yaml)	js-yaml + Parser	Bóc tách cấu trúc thành bảng tổng quan Endpoints (METHOD, PATH, Summary) và bảng chi tiết Parameters, Responses.
Postman Collection (.json)	JSON Parser	Trích xuất danh mục Requests có phân nhóm thư mục logic.
Plain Text (.txt, .sql)	Native Node.js	Chuẩn hóa định dạng UTF-8, dọn dẹp khoảng trắng thừa.

Bảng 3.1: Năng lực chuyển đổi của bộ công cụ Universal Document Converter

CHƯƠNG 4

BỘ NÃO TRI THỨC (SSOT KNOWLEDGE)

4.1. Nguyên Lý Nguồn Sự Thật Duy Nhất (Single Source of Truth)

Một trong những thách thức lớn nhất của các dự án phần mềm là sự trôi dạt thông tin (knowledge drift) và hiện tượng quên ngữ cảnh (context loss) khi bàn giao giữa các thành viên hoặc giữa các phiên làm việc của AI Agent. Khi một câu hỏi nghiệp vụ đã được BA trả lời trong một phiên họp, nếu thông tin đó chỉ nằm lại trong kênh chat hoặc biên bản họp, những người khác (hoặc các AI Agent ở lượt chạy sau) sẽ lặp lại chính câu hỏi đó hoặc hiểu sai logic kiểm thử.

Thư mục knowledge/ đóng vai trò là Bộ Não Tri Thức Vĩnh Viễn của dự án, tuân thủ nguyên tắc SSOT:

- **Tính kế thừa:** Mọi quy tắc nghiệp vụ đã chốt từ các tài liệu trước hoặc các phiên Q&A với BA/PO đều được ghi nhận vào đây.
- **Cơ sở FACT 100%:** Mọi kết luận trong các bản phân tích rủi ro, danh mục viewpoint và đặc tả Test Case đều phải dẫn xuất từ knowledge/. Agent bị nghiêm cấm việc tự suy diễn quy tắc nghiệp vụ nếu chưa có cơ sở từ SSOT.
- **Loại bỏ việc hỏi lại:** Khi một kẻ hở logic nghiệp vụ được giải quyết, trạng thái câu hỏi được cập nhật từ New sang Confirmed. Ở các lượt chạy tiếp theo, Agent tự động áp dụng quy tắc này mà không bao giờ hỏi lại người dùng.

4.2. Cấu Trúc Thư Mục knowledge/

[Cấu trúc tệp tin trong knowledge/ (text)]

knowledge/

- _project.md <- Quy ước chung toàn hệ thống (Tiền tệ, Múi giờ, Định dạng mã)
- _glossary.md <- Bảng thuật ngữ chuẩn hóa viết tắt & định nghĩa nghiệp vụ
- _system_map.json <- Bản đồ định tuyến tính năng & chỉ mục vị trí tệp tin
- features/ <- Kho tri thức riêng của từng tính năng
 - checkout.md
 - voucher.md
 - ...

4.2.1. _project.md – Quy Ước Chung Toàn Hệ Thống

Tệp tin này định nghĩa các chuẩn mực kỹ thuật và quy định nghiệp vụ cốt lõi xuyên suốt toàn bộ dự án:

- **Tiền tệ:** Đơn vị tiền tệ chuẩn (VND, USD), quy tắc làm tròn (làm tròn số học đến hàng đơn vị).
- **Múi giờ & Thời gian:** Múi giờ hệ thống (GMT+7 / Asia/Ho_Chi_Minh), lưu trữ DB (UTC ISO-8601), hiển thị UI (DD/MM/YYYY HH:mm:ss).
- **Định dạng mã:** Quy chuẩn đặt mã định danh (Đơn hàng ORD-YYYYMMDD-XXXXX, Mã voucher VCH-[A-Z0-9]{6,12}).
- **Môi trường kiểm thử:** Base URL cho các môi trường STG/UAT, tài khoản kiểm thử mặc định theo vai trò (Buyer, Seller, Admin).

4.2.2. _glossary.md – Từ Điển Thuật Ngữ Chuẩn Hóa

Thuật ngữ / Viết tắt

Tên tiếng Anh

Định nghĩa chuẩn hóa trong hệ thống

AOV	Average Order Value	Giá trị đơn hàng trung bình = Tổng doanh thu / Số đơn hàng thành công.
COD	Cash on Delivery	Hình thức thanh toán tiền mặt khi nhận hàng.
Stacking Voucher	Voucher Stacking	Quy tắc áp dụng đồng thời nhiều voucher khác loại trên một đơn hàng.
Hold Time	Inventory Hold	Thời gian giữ tồn kho khi người dùng nhấn đặt hàng (mặc định: 15 phút).

Bảng 4.1: Ví dụ trích xuất từ bảng từ điển thuật ngữ `_glossary.md`

4.2.3. `_system_map.json` – Bản Đồ Định Tuyến Vệ Tinh (System Map)

Để chống hiện tượng Agent đọc dò tệp tin (blind scanning) làm tiêu tốn hàng nghìn token và làm đầy context window, `_system_map.json` cung cấp một bảng chỉ mục tính được đồng bộ liên tục:

[Cấu trúc dữ liệu của `_system_map.json` (json)]

```
{
  "project_name": "E-Commerce Enterprise Platform",
  "version": "1.0.0",
  "last_synced": "2026-09-18T08:00:00.000Z",
  "routing_table": {
    "voucher-stacking": {
      "knowledge_file": "knowledge/features/voucher-stacking.md",
      "input_dir": "INPUT/voucher-stacking/",
      "output_dir": "OUTPUT/voucher-stacking/",
      "status": "IN-PROGRESS",
      "latest_milestone": "04_test_idea_report.md",
      "rule_count": 12,
      "unresolved_questions": 2
    }
  }
}
```

4.3. Vòng Đời Tri Thức Tính Năng (Feature Knowledge Lifecycle)

⚡ VÒNG ĐỜI TIẾN HÓA CỦA TRI THỨC TÍNH NĂNG

[GIAI ĐOẠN 1] Khám Phá (Discovery): Đọc tài liệu thô trong INPUT/ và quét kể hờ logic O6W.



[PHÂN NHÁNH] Tách 2 Dòng Thông Tin: Quy tắc rõ ràng -> Confirmed Rules (BR-xx). Kể hờ logic -> Open Questions (Verdict: ASK).



[GIAI ĐOẠN 2] Tham Vấn BA / PO: Chuyển danh sách câu hỏi mở cho BA để chốt logic chính thức.



[GIAI ĐOẠN 3] Tiến Hành Thiết Kế Kiểm Thử: Sinh Viewpoints, Test Ideas, Test Cases chuẩn FACT dựa trên các quy tắc đã xác nhận.



[GIAI ĐOẠN 4] Kế Thừa & Tích Lũy SSOT: Tự động khóa câu hỏi (Confirmed), tích lũy vào kho tri thức và không bao giờ hỏi lại.

Hình 4.1: Sơ đồ vòng đời tiến hóa của tri thức tính năng trong hệ thống SSOT

4.4. Cấu Trúc Tập Tri Thức Tính Năng features/<slug>.md

Một tập tri thức tính năng tiêu chuẩn gồm 8 phần bắt buộc:

- **1. Bối Cảnh Tính Năng (Context & Scope):** Mục tiêu kinh doanh, đối tượng sử dụng, ranh giới In-Scope và Out-of-Scope.
- **2. Sơ Đồ Luồng Nghiệp Vụ (Business Workflow):** Trình bày tuần tự các bước người dùng thao tác hoặc hệ thống xử lý.
- **3. Confirmed Business Rules (Quy Tắc Đã Xác Nhận):** Danh sách các quy tắc có trích dẫn nguồn rõ ràng từ SRS/PRD. Mỗi quy tắc có mã định danh BR-xx.
- **4. Data Entities & Constraints:** Kiểu dữ liệu, độ dài, khoảng giá trị hợp lệ, các giá trị biên của từng trường.
- **5. API & Integration Contracts:** Endpoints, Headers, Request Body, Response Status Code, Error Codes từ tài liệu Dev/Swagger.
- **6. UI/UX Specifications:** Trạng thái hiển thị (Enable, Disable, Hidden, Loading), thông điệp lỗi tương ứng với Figma Design.
- **7. Open Questions (Các Câu Hỏi Mở Đang Chờ BA):** Những kẻ hở logic được phát hiện qua bộ lọc O6W nhưng chưa có câu trả lời. Ghi rõ mức độ nghiêm trọng và tác động.
- **8. Giả Định Đã Chốt (Resolved Assumptions):** Nhật ký ghi nhận các câu trả lời từ BA/PO kèm ngày chốt và người phê duyệt, là căn cứ để chuyển đổi câu hỏi mở thành BR-xx.

4.5. Lệnh Tiện Ích Quản Trị Tri Thức

```
[Lệnh quản trị tri thức (bash)]
npm run knowledge:new <feature-slug>
npm run map:sync
```

CHƯƠNG 5: MÔ HÌNH ĐIỀU PHỐI CỦA QA LEADER

5.1. Nguyên Tắc Tổng Chỉ Huy Một Cửa (Single Point of Contact)

Trong các hệ sinh thái Multi-Agent thông thường, người dùng thường phải đối mặt với sự phức tạp khi phải nhớ tên từng sub-agent, cú pháp triệu gọi, cũng như tự mình chuyển giao dữ liệu đầu ra của Agent này làm đầu vào cho Agent khác. Điều này gây lãng phí thời gian, dễ nhầm lẫn và đòi hỏi người dùng phải có chuyên môn kỹ thuật sâu.

Kiến trúc điều phối của hệ thống giải quyết triệt để vấn đề trên thông qua cơ chế QA Leader One-Stop-Shop:

- **Giao diện duy nhất:** Người dùng (Tester, BA, PO, Manager) CHỈ CẦN TƯƠNG TÁC VỚI QA LEADER. Mọi yêu cầu bằng ngôn ngữ tự nhiên đều được QA Leader tiếp nhận và xử lý.
- **Tự động phân giải ý định (Intent Parsing):** QA Leader phân tích câu lệnh của người dùng để xác định xem đang ở giai đoạn nào (Phân tích rủi ro, Sinh Test Case, Chuẩn bị dữ liệu hay Chạy thực thi kiểm thử).
- **Điều động chuyên gia tự động:** QA Leader trực tiếp gọi các sub-agent chuyên trách thông qua các kỹ năng (Skills) nội bộ, truyền đúng ngữ cảnh và kiểm soát chất lượng đầu ra trước khi phản hồi người dùng.

5.2. Sơ Đồ Tuần Tự Điều Phối Hệ Thống

⚡ QUY TRÌNH TUẦN TỰ ĐIỀU PHỐI TÁC VỤ CỦA QA LEADER

[BUỐC 1] User Yêu Cầu Tự Nhiên: Người dùng nhập yêu cầu bằng tiếng Việt (ví dụ: 'Tạo test case cho tính năng voucher').



[BUỐC 2] QA Leader Tra Cứu Bản Đồ: QA Leader đọc `_system_map.json` để lấy đường dẫn chính xác của tính năng.



[BUỐC 3] Đọc Kế Hoạch `00_plan.md`: QA Leader kiểm tra chặng đang dở và nạp đúng các tài liệu đầu vào tương ứng.



[BUỐC 4] Giao Việc Kèm Skill: QA Leader triệu gọi Sub-Agent (Analyst, Test Design...) kèm kỹ năng chuyên sâu và nhận bản thảo.



[BUỐC 5] Kiểm Định Cổng Verdict Gate: Đánh giá kết quả theo 3 trạng thái PASS, FIX hoặc ASK.



[BUỐC 6] Cập Nhật Kế Hoạch & Báo Cáo: Đánh dấu hoàn thành [X] trong `00_plan.md` và phản hồi bảng Dashboard trực quan cho người dùng.

Hình 5.1: Sơ đồ trực quan tuần tự điều phối tác vụ của QA Leader

5.3. Cơ Chế Kiểm Soát Bằng `00_plan.md`

Mọi hoạt động của QA Leader đều xoay quanh tệp điều khiển trung tâm: `OUTPUT/<task-slug>/00_plan.md`.

- **Phân mảnh ngữ cảnh (Context Isolation):** Thay vì nạp toàn bộ lịch sử trò chuyện dài hàng chục ngàn từ, mỗi khi thực thi một chặng, QA Leader chỉ đọc trạng thái hiện tại của `00_plan.md` và các tệp đầu vào trực tiếp của chặng đó.

- **Cầu nối đa nền tảng (Cross-Tool Continuity):** Người dùng có thể bắt đầu tác vụ trên Antigravity IDE, sau đó chuyển sang Claude Code hoặc Cursor IDE. Agent mới chỉ cần mở OO_plan.md là tiếp tục chính xác vị trí công việc mà không cần giải thích lại bối cảnh.

[Cấu trúc tệp tin OUTPUT/<task-slug>/OO_plan.md (markdown)]

Kế Hoạch Phân Tích & Kiểm Thử · voucher-stacking

Ngày tạo: 2026-09-18 · Người lập: QA Leader · Trạng thái: IN-PROGRESS

1. Phạm Vi & Tài Liệu Nguồn

- Tài liệu yêu cầu: INPUT/voucher-stacking/O2_ba/srs_voucher_v1.2.md
- Tri thức dự án: knowledge/_project.md, knowledge/_glossary.md
- Tri thức tính năng: knowledge/features/voucher-stacking.md

2. Lộ Trình Từng Chặng (Milestones & Explicit Skills)

- [X] Chặng 1: Đọc yêu cầu thô & Phân tích rủi ro [qa-analyst/requirement-risk-summary] → Ra 01_requirement_risk_summary.md (Verdict: PASS)
- [X] Chặng 2: Quét kẽ hở O6W & Câu hỏi cho BA [qa-analyst/missing-rule-O6w] → Ra 02_missing_rule_report.md (Verdict: PASS - Đã chốt 3 giả định)
- [] Chặng 3: Chọn Risk Area & Viewpoints [qa-analyst/viewpoint-selection] → Ra 03_viewpoint_report.md (Đang thực hiện)
- [] Chặng 4: Thiết kế Test Idea & Lọc Giữ/Bỏ [qa-analyst/test-idea-design] → Ra 04_test_idea_report.md
- [] Chặng 5: Sinh Test Case chi tiết 8 trường [qa-test-design/test-case-generation] → Ra 05_test_case_spec.md
- [] Chặng 6: Rà soát độ phủ 3 góc nhìn & Nghiệm thu [qa-test-design/coverage-review] → Ra 06_coverage_review.md

5.4. Hệ Thống 3 Cổng Đánh Giá Chất Lượng (Verdict Gates)

Trạng thái Gate	Điều kiện kích hoạt	Hành vi xử lý của QA Leader
PASS	Đạt 100% tiêu chuẩn FACT, đầy đủ mã định danh, không có mâu thuẫn logic.	Đánh dấu hoàn thành chặng trong OO_plan.md và tự động kích hoạt chặng kế tiếp.
FIX	Thiếu sót định dạng, sai sót mã truy vết (traceability), vi phạm cấu trúc trường.	Tự động gửi phản hồi chi tiết cho Sub-Agent kèm danh sách lỗi để sửa lại (Self-Correction), không làm phiền người dùng.
ASK	Phát hiện lỗ hổng nghiệp vụ, thiếu quy định xử lý biên, xung đột giữa các tài liệu.	LẬP TỨC DỪNG QUY TRÌNH. Liệt kê danh sách câu hỏi cụ thể kèm mức độ rủi ro để người dùng chuyển tới BA/PO.

Bảng 5.1: Quy định hành vi của QA Leader đối với 3 Verdict Gates

CHƯƠNG 6

QUY TRÌNH THIẾT KẾ KIỂM THỬ CHUẨN 6 CHẶNG

6.1. Tổng Quan Khung Quy Trình 6 Chặng (Milestones 01 -> 06)

Khác với các phương pháp kiểm thử truyền thống thường nhảy vọt từ yêu cầu sơ khai sang viết kịch bản kiểm thử, kiến trúc kiểm thử chuyên sâu của hệ thống áp dụng chu trình phân tách đa tầng 6 chặng. Quy trình này đảm bảo kiểm soát chặt chẽ từng bước chuyển đổi logic, ngăn chặn sai sót tích lũy và phát hiện sớm các rủi ro tiềm ẩn ngay từ giai đoạn tài liệu.

⚡ QUY TRÌNH THIẾT KẾ KIỂM THỬ CHUẨN 6 CHẶNG

[CHẶNG 1] Phân Tích Yêu Cầu & Rủi Ro: Trích xuất Confirmed Business Rules (BR-xx), ranh giới In/Out Scope, xếp hạng rủi ro -> 01_requirement_risk_summary.md.

↓
[CHẶNG 2] Quét Kẽ Hở Logic 06W: Kiểm tra 6 câu hỏi W1..W6 tìm điểm tài liệu chưa nói. Nếu có câu hỏi trọng yếu -> DỪNG LẠI (Verdict: ASK) -> 02_missing_rule_report.md.

↓
[CHẶNG 3] Lựa Chọn Góc Nhìn Kiểm Thử: Bao phủ đa chiều theo chuẩn ISO 29119: Functional, Data, Lifecycle, Security, Concurrency -> 03_viewpoint_report.md.

↓
[CHẶNG 4] Thiết Kế Test Idea & Lọc Giữ/Bỏ: Phối hợp Rules và Viewpoints sinh ý tưởng, lọc KEEP (rủi ro cao) vs DROP (trùng lặp/chi phí lớn) -> 04_test_idea_report.md.

↓
[CHẶNG 5] Đặc Tả Test Case Chuẩn 8 Trường: Sinh chi tiết 8 trường chuẩn FACT 100%, áp dụng Blueprint Batching chống tràn token -> 05_test_case_spec.md.

↓
[CHẶNG 6] Rà Soát Độ Phủ 3 Chiều & Nghiệm Thu: Đánh giá Rule Coverage (100%), Viewpoint Coverage và Risk Coverage -> 06_coverage_review.md.

Hình 6.1: Sơ đồ trực quan quy trình thiết kế kiểm thử 6 chặng

6.2. Chi Tiết Từng Chặng Trong Pipeline

Chặng 1: Tóm Tắt Yêu Cầu & Đánh Giá Rủi Ro (01)

- **Mục tiêu:** Đọc tài liệu yêu cầu thô từ INPUT/, phân định rõ ràng các ranh giới nghiệp vụ và xác định trọng tâm kiểm thử.
- **Sản phẩm đầu ra:** 01_requirement_risk_summary.md.
- **Nội dung cốt lõi:** Confirmed Business Rules (đánh số BR-01, BR-02... kèm trích dẫn nguồn), Phân định In-Scope vs Out-of-Scope, Xếp loại rủi ro (Critical, High, Medium, Low).

Chặng 2: Quét Kẽ Hở Logic & Khảo Sát 06W (02)

Bộ lọc 06W chuẩn mực quét tìm những trường hợp ngoại lệ hoặc mâu thuẫn ngầm định:

- **W1 (Who):** Ai thực hiện? Phân quyền? Tài khoản bị khóa, đổi mật khẩu giữa chừng?
- **W2 (What):** Dữ liệu đầu vào là gì? Rỗng, cực đại, ký tự đặc biệt UTF-8, SQL Injection, XSS?
- **W3 (When):** Thời điểm nào? Hết hạn đúng 00:00:00, lệch múi giờ Client/Server, race condition?
- **W4 (Where):** Môi trường ở đâu? Mạng chập chờn (flaky network), mất kết nối đúng lúc Submit, chuyển tab?

- **W5 (Why):** Lý do nghiệp vụ? Tại sao người dùng hủy đơn, hoàn tiền, tại sao voucher bị vô hiệu hóa?
- **W6 (How & What-If):** Hệ thống xử lý thế nào khi Third-Party (Cổng thanh toán, Vận chuyển) bị timeout?

Chặng 3: Lựa Chọn Góc Nhìn Kiểm Thử (03)

Tránh lối mòn chỉ kiểm thử chức năng đơn thuần bằng cách bao phủ đa dạng các nhóm Viewpoint theo ISO 29119:

- **Functional Viewpoint:** Luồng chính (Happy Path), Luồng phụ (Alternative Path), Luồng lỗi (Negative Path).
- **Data Viewpoint:** Giá trị biên (Boundary), Lớp tương đương (Equivalence Partitioning), Định dạng đặc biệt.
- **Lifecycle & State Viewpoint:** Chuyển đổi trạng thái thực thể (Tạo mới -> Chờ duyệt -> Kích hoạt -> Khóa).
- **Security & Authority Viewpoint:** Leo thang đặc quyền, giả mạo tham số (Tampering), CSRF, Token hết hạn.
- **Concurrency & Performance Viewpoint:** Thao tác đồng thời từ 2 phiên đăng nhập, nhấp đúp liên tục (double clicking).

Chặng 4: Thiết Kế Ý Tưởng & Lọc Giữ/Bỏ (04)

- **Tiêu chí KEEP:** Các ca kiểm thử bao phủ rủi ro cao, liên quan đến tiền tệ, bảo mật, hoặc các luồng người dùng cốt lõi.
- **Tiêu chí DROP:** Các ca kiểm thử trùng lặp logic, kiểm thử bên thứ ba đã kiểm định riêng, hoặc kịch bản có xác suất xảy ra cực thấp với chi phí tạo dữ liệu quá lớn. Mọi quyết định DROP đều ghi rõ lý do.

Chặng 5: Đặc Tả Test Case Chi Tiết Chuẩn 8 Trường (05)

Mỗi Test Case trong O5_test_case_spec.md bắt buộc phải có đầy đủ 8 trường thông tin chuẩn mực:

STT	Tên trường	Đặc tả chi tiết & Yêu cầu
1	Test Case ID	Định danh duy nhất theo quy ước: TC_<MODULE>_<SO_THU_TU> (ví dụ: TC_VCH_001).
2	Module / Feature	Tên phân hệ hoặc tính năng kiểm thử.
3	Rule ID	Mã định danh quy tắc nghiệp vụ mà Test Case này kiểm chứng (BR-01, BR-02...).
4	Viewpoint ID	Mã định danh góc nhìn kiểm thử (VP-01, VP-SEC-02...).
5	Mục Đích Kiểm Thử	Mô tả ngắn gọn một câu: Kiểm chứng điều gì trong điều kiện nào.
6	Tiền Điều Kiện	Trạng thái môi trường, tài khoản đăng nhập, dữ liệu có sẵn trước khi thực hiện.
7	Các Bước Thực Hiện	Các thao tác người dùng theo từng bước cụ thể (Step 1, Step 2...).
8	Kỳ Vọng Đầu Ra	Kết quả quan sát được một cách nhị phân (Pass/Fail): UI, thông báo, thay đổi DB hoặc mã phản hồi API.

Bảng 6.1: Quy chuẩn 8 trường thông tin của một Test Case

Chặng 6: Rà Soát Độ Phủ 3 Chiều & Nghiệm Thu (06)

- **Rule Coverage:** 100% các quy tắc BR-xx phải có ít nhất một Test Case bao phủ.

- **Viewpoint Coverage:** Đảm bảo không bị thiên lệch (ví dụ: chỉ có Happy Path mà không có Security hay Concurrency).
- **Risk Coverage:** 100% các rủi ro mức Critical và High phải có kịch bản tương ứng.

6.3. Tầng Thực Thi Độc Lập (OUTPUT/<slug>/runs/)

Để giữ cho tài liệu thiết kế gốc (Master Spec) luôn sạch sẽ và không bị xáo trộn qua các chu kỳ kiểm thử thực tế, toàn bộ kết quả chạy được cô lập tại tầng runs/:

[Cấu trúc phiên thực thi tại tầng runs/ (text)]

OUTPUT/<task-slug>/runs/RUN-01_ticket-102/

- └─ run_plan.md <- Danh sách trích xuất 15 Test Cases được chọn chạy
- └─ run_result.md <- Bảng kết quả thực tế (Pass/Fail/Blocked) kèm timestamp
- └─ run_defects.md <- Báo cáo lỗi chi tiết phát hiện trong phiên
- └─ evidence/ <- Ảnh chụp màn hình và log bằng chứng thực tế
 - └─ TC_001_pass.png
 - └─ TC_008_error.png

CHƯƠNG 7

XỬ LÝ QUY MÔ LỚN & ĐỘNG CƠ SINH DỮ LIỆU TỰ ĐỘNG

7.1. Thách Thức Tràn Token Khi Kiểm Thử Quy Mô Lớn

Trong các hệ thống phần mềm doanh nghiệp, một phân hệ phức tạp thường đòi hỏi từ hàng trăm đến hàng nghìn kịch bản kiểm thử chi tiết để bao phủ toàn bộ ma trận dữ liệu và các hoán vị điều kiện biên. Nếu Agent cố gắng sinh toàn bộ hàng trăm Test Cases trong một lượt phản hồi duy nhất (One-shot generation):

- **Cắt cụt Token (Truncation):** Giới hạn token xuất tối đa (Max Output Tokens) thường dừng ở 4.096 đến 8.192 tokens. Khi vượt quá giới hạn, phản hồi sẽ bị đứt gãy giữa chừng.
- **Suy giảm chất lượng (Degradation):** Càng về cuối phản hồi dài, mô hình LLM có xu hướng rút ngắn các bước thực hiện, bỏ qua tiền điều kiện hoặc đưa ra kết quả kỳ vọng chung chung, vi phạm tiêu chuẩn FACT.
- **Chi phí Token khổng lồ:** Việc lặp lại các đoạn văn bản dài tiêu tốn một lượng token lớn không cần thiết.

⚡ QUY TRÌNH 3 GIAI ĐOẠN XỬ LÝ QUY MÔ LỚN

[GIAI ĐOẠN 1] Bản Thiết Kế Khung (Blueprint JSON): Xuất O5_test_blueprint.json (~30 tokens/mục cho 500-1.000 TCs), bao quát 100% phạm vi logic mà không bị tràn context.



[GIAI ĐOẠN 2] Chia Lô Sinh Chi Tiết (Chunked Batching): Chia danh mục thành từng lô: 50 Test Cases / Batch (batch_01.md, batch_02.md...). Mỗi ca đủ 8 trường chuẩn FACT.



[GIAI ĐOẠN 3] Gộp Tổng Thể Tự Động (Assembly): Kích hoạt script merge-testcases.js tự động ghép tất cả các batch thành O5_test_case_spec.md hoàn chỉnh.



[BỔ TRỢ] Data Engine Tốc Độ Cao (0 Token LLM): Định nghĩa dataset_schema.json siêu nhẹ, chạy generate-dataset.js sinh 1.000 dòng trong 0.05s vào 10_dataset.md.

Hình 7.1: Sơ đồ trực quan 3 giai đoạn xử lý kiểm thử quy mô lớn và động cơ dữ liệu

7.2. Giai Đoạn 1: Bản Thiết Kế Khung (O5_test_blueprint.json)

[Cấu trúc tệp tin O5_test_blueprint.json (json)]

```
[
  {
    "id": "TC_VCH_001",
    "module": "Voucher Application",
    "rule_id": "BR_01",
    "viewpoint_id": "VP_HAPPY_01",
    "objective": "Áp dụng thành công 1 voucher hợp lệ với đơn hàng đạt giá trị tối thiểu",
    "priority": "P1"
  },
  {
    "id": "TC_VCH_002",
    "module": "Voucher Stacking",
    "rule_id": "BR_02",
    "viewpoint_id": "VP_BOUNDARY_01",
    "objective": "Áp dụng đồng thời Voucher Freeship và Voucher Giảm giá của Shop",
    "priority": "P1"
  }
]
```

```
}  
]
```

Với cấu trúc gọn nhẹ này, Agent có thể định hình bản đồ của 500 đến 1.000 ca kiểm thử chỉ trong 15.000 tokens mà không gặp bất kỳ hiện tượng quá tải nào.

7.3. Giai Đoạn 2: Chia Lô Sinh Chi Tiết (Chunked Batching)

- Lô 1: OUTPUT/<slug>/testcases/batch_01.md (TC-001 đến TC-050)
- Lô 2: OUTPUT/<slug>/testcases/batch_02.md (TC-051 đến TC-100)
- Lô N: OUTPUT/<slug>/testcases/batch_NN.md

7.4. Giai Đoạn 3: Gộp Tự Động (Assembly)

```
[Lệnh gộp test cases (bash)]  
npm run testcases:merge <task-slug>
```

7.5. Động Cơ Sinh Dữ Liệu Tự Động (0-Token LLM Data Engine)

Một sai lầm phổ biến khi sử dụng AI trong kiểm thử là yêu cầu LLM 'gõ tay' từng dòng dữ liệu test (ví dụ: yêu cầu sinh 100 tên người dùng, 100 số điện thoại và địa chỉ). Việc này cực kỳ lãng phí token, sinh dữ liệu chậm và dễ bị trùng lặp. Kiến trúc hệ thống sử dụng Data Generator Engine cục bộ (agents/tools/generate-dataset.js):

- **LLM chỉ định nghĩa Schema:** AI Agent chỉ cần xuất một tệp dataset_schema.json siêu nhẹ (~40 token) định nghĩa cấu trúc cột và kiểu dữ liệu.
- **Tốc độ tức thì (0.05s):** Script Node.js nội bộ sinh hàng trăm/ngày nghìn dòng dữ liệu chuẩn xác trong 0.05s mà không tiêu tốn một token LLM nào.

Tên Generator	Tham số cấu hình	Ví dụ kết quả sinh ra
vietnamese_name	{"gender": "any male female"}	Nguyễn Văn Hùng, Trần Thị Mai
phone_vn	{"carrier": "viettel vinaphone any"}	0981234567, 0918765432
email	{"domain": "gmail.com test.vn"}	hung.nguyen482@test.vn
voucher_code	{"prefix": "SALE", "len": 8}	SALE-9X8B7C, SALE-KJ4M2Q
currency_vnd	{"min": 50000, "max": 2000000}	150.000 đ, 1.250.000 đ
date_vn	{"format": "DD/MM/YYYY", "past": false}	25/12/2026, 01/01/2027
boundary	{"min": 1, "max": 100, "type": "int"}	0, 1, 2, 99, 100, 101
enum	{"values": ["COD", "MOMO", "BANK"]}	Lấy ngẫu nhiên hoặc tuần tự theo mảng
negative	{"rule": "special_chars sql xss"}	<script>alert(1)</script>, ' OR '1'='1

Bảng 7.1: Danh mục các Generator có sẵn trong công cụ nội bộ

```
[Tệp tin cấu hình dataset_schema.json (json)]  
{  
  "dataset_name": "voucher_stress_testing",  
  "fields": [  

```

```
{ "name": "customer_name", "type": "vietnamese_name" },  
{ "name": "phone_number", "type": "phone_vn", "carrier": "viettel" },  
{ "name": "order_amount", "type": "currency_vnd", "min": 100000, "max": 5000000 },  
{ "name": "voucher_code", "type": "voucher_code", "prefix": "FLASH" },  
{ "name": "payment_method", "type": "enum", "values": ["COD", "VNPAY", "MOMO"] }  
]  
}
```

CHƯƠNG 8

KỊCH BẢN VẬN HÀNH 100% NGÔN NGỮ TỰ NHIÊN

8.1. Triết Lý Trải Nghiệm Zero-CLI



ĐIỀU KHOẢN TRẢI NGHIỆM NGƯỜI DÙNG TỐI CAO

Người dùng KHÔNG BIẾT VÀ KHÔNG PHẢI GỖ BẤT KỲ LỆNH TERMINAL NÀO.

Mọi AI Agent bị nghiêm cấm việc hướng dẫn người dùng: 'Bạn hãy mở terminal và gõ lệnh...'. Mọi kịch bản tương tác đều diễn ra bằng tiếng Việt tự nhiên trong khung chat. AI Agent tự động chạy các script nội bộ ở hậu trường và chỉ phản hồi kết quả trực quan.

8.2. Sơ Đồ Phân Giải Ý Định Tự Nhiên Của QA Leader



CƠ CHẾ PHÂN GIẢI Ý ĐỊNH NGÔN NGỮ TỰ NHIÊN

[INPUT] Người Dùng Nhập Câu Lệnh Tự Nhiên: 'Tiến độ thế nào?' • 'Có file mới' • 'BA đã chốt' • 'Cần 50 data test'



[INTENT PARSER] QA Leader Phân Giải Ý Định: Tự động nhận diện nghiệp vụ, tra cứu bản đồ hệ thống, xác định công cụ tương ứng.



[BACKGROUND SCRIPT] Kích Hoạt Công Cụ Chạy Ngầm: Tự chạy status.js / intake.js / sync-system-map.js / generate-dataset.js ở hậu trường.



[OUTPUT] Phản Hồi Bảng Dashboard Trực Quan: Trả kết quả tiếng Việt kèm đề xuất bước đi tiếp theo (Zero-Path Typing).

Hình 8.1: Sơ đồ trực quan cơ chế phân giải ý định ngôn ngữ tự nhiên thành hành động kỹ thuật

8.3. Bốn Kịch Bản Đối Thoại Điển Hình



Kịch Bản 1: Tiếp Tục Tác Vụ Đang Dở (Zero-Path Typing)



Người dùng: 'Hôm nay làm gì tiếp nhỉ?'



QA Leader:

'Chào bạn! Tôi đã kiểm tra toàn bộ không gian làm việc của dự án. Dưới đây là tiến độ hiện tại:

- voucher-stacking: Đã xong Chặng 2 (02_missing_rule), Chặng tiếp theo: Chặng 3 (Viewpoints) - Đang chờ BA.
- checkout-flow: Đã xong Chặng 4 (Test Ideas), Chặng tiếp theo: Chặng 5 (Test Cases) - Sẵn sàng sinh.

Tính năng voucher-stacking hiện đang có 2 câu hỏi cần làm rõ với BA. Trong khi chờ đợi, tôi đề xuất chúng ta tiếp tục sinh Test Cases cho checkout-flow (Lô 1: 50 cases đầu tiên). Bạn có muốn tôi tiến hành ngay không?'



Kịch Bản 2: Tiếp Nhận Phản Hồi Từ BA & Mở Khóa Quy Trình



Người dùng: 'BA vừa chốt rồi nhé: Nếu 2 voucher cùng loại giảm giá %, hệ thống sẽ chỉ lấy voucher có giá trị giảm cao nhất, không cộng dồn.'



QA Leader:

'Rất tuyệt vời! Tôi đã cập nhật thông tin này vào Bộ Não Tri Thức:

1. Cập nhật SSOT: Ghi nhận quy tắc mới BR-VCH-09 vào knowledge/features/voucher-stacking.md.
2. Đóng câu hỏi mở: Chuyển trạng thái câu hỏi Q-02 từ New → Confirmed.
3. Cập nhật Kế hoạch: Đánh dấu Cổng Chặng 2 chuyển từ ASK → PASS.

Rào cản nghiệp vụ đã được gỡ bỏ. Tôi sẽ kích hoạt chuyên gia Test Design để chuyển sang Chặng 3: Lựa chọn Viewpoints ngay bây giờ!



Kịch Bản 3: Tiếp Nhận Tài Liệu Thay Đổi (Change Request - CR)



Người dùng: 'Tôi vừa thả file SRS mới sửa của BA vào thư mục INPUT, kiểm tra giúp tôi với.'



QA Leader:

'Tôi đã nhận diện tệp mới srs_voucher_v2.0.docx trong thư mục INPUT/voucher-stacking/02_ba/. Tôi đã tự động chuyển đổi sang Markdown và so sánh với phiên bản cũ:

- Thay đổi phát hiện: Quy tắc hạn mức thanh toán tối thiểu đã tăng từ 200.000 đ lên 300.000 đ (Ảnh hưởng BR-03).
- Phân tích bán kính ảnh hưởng (Blast Radius): Có 6 Test Cases hiện tại (TC-012 đến TC-017) đang sử dụng giá trị biên 200.000 đ cũ.

Tôi đề xuất cập nhật lại 6 Test Cases này để khớp với SRS mới. Bạn có đồng ý cập nhật không?



Kịch Bản 4: Khởi Tạo Phiên Thực Thi Kiểm Thử Trên Môi Trường



Người dùng: 'Chạy test cho Ticket BUG-892 trên môi trường Staging nhé: <https://staging.myshop.vn>'



QA Leader:

'Đã rồi! Tôi khởi tạo phiên thực thi độc lập tại:
OUTPUT/voucher-stacking/runs/RUN-02_BUG-892/

- Phạm vi kiểm thử: Trích xuất 10 Test Cases liên quan đến luồng áp dụng mã giảm giá giỏ hàng từ Master Spec.
- Môi trường: <https://staging.myshop.vn>.

Sau khi thực thi kiểm thử với chuyên gia Automation, tôi sẽ ghi nhận kết quả vào run_result.md, chụp ảnh màn hình lưu vào evidence/ và lập báo cáo lỗi run_defects.md nếu phát hiện sai lệch.'

CHƯƠNG 9

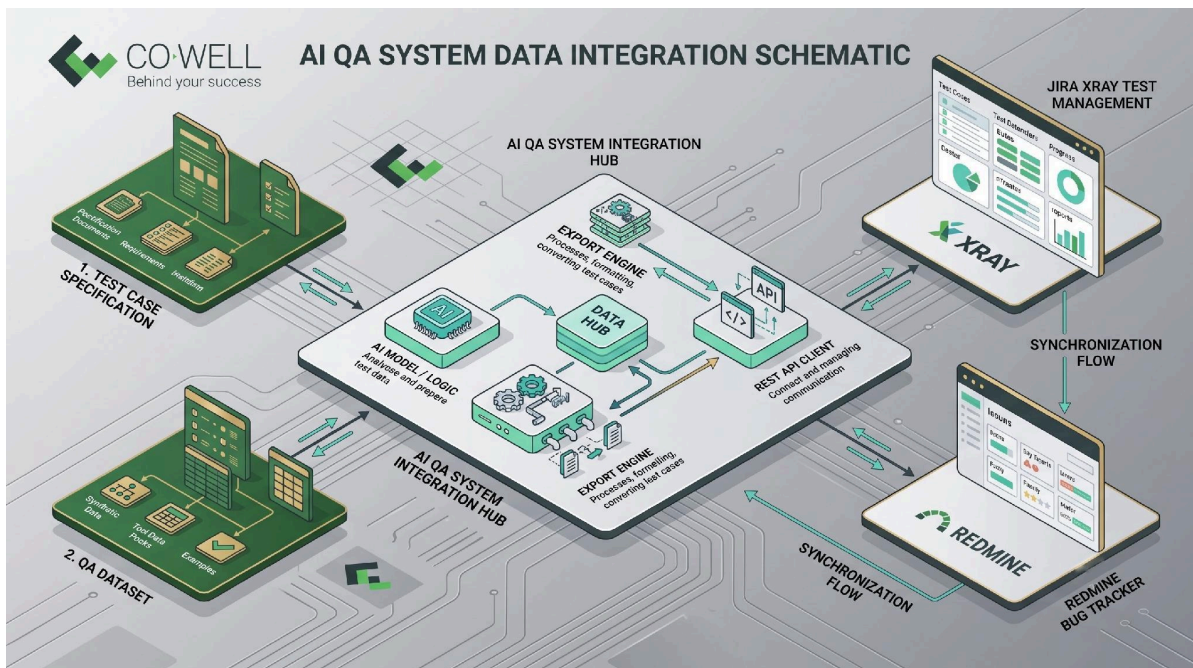
TÍCH HỢP QUẢN LÝ KIỂM THỬ (JIRA / REDMINE)

9.1. Tổng Quan Nhu Cầu Tích Hợp Doanh Nghiệp

Trong các môi trường doanh nghiệp thực tế, các bộ phận kiểm thử hiếm khi làm việc độc lập với các tệp Markdown trên máy cục bộ. Kết quả thiết kế kiểm thử, dữ liệu và các lỗi phát hiện cần được đồng bộ lên các nền tảng quản lý dự án và kiểm thử tiêu chuẩn như Atlassian Jira (kèm plugin Xray / Zephyr) hoặc Redmine.

Hệ thống cung cấp cơ chế tích hợp kép linh hoạt:

- 1. **Đồng bộ trực tiếp qua REST API (2-Way Direct Sync):** Tự động đẩy Test Cases, Test Execution và Defect lên server qua token xác thực.
- 2. **Xuất tệp trao đổi chuẩn hóa (Standard CSV/JSON Export):** Hỗ trợ mã hóa UTF-8 có BOM (Byte Order Mark) giúp mở trực tiếp trên Microsoft Excel và Jira Importer mà không bị lỗi hiển thị tiếng Việt có dấu.



Hình 9.1: Sơ đồ kiến trúc tích hợp đồng bộ giữa Hệ thống Agent và Jira / Redmine

9.2. Ánh Xạ Dữ Liệu (Field Mapping)

Ánh Xạ Sang Jira Xray Test Issue

Trường Trong Master Spec	Trường Trong Jira Xray	Quy cách chuyển đổi
Test Case ID	Test Case Key / Summary Prefix	Gắn mã định danh vào đầu Summary: [TC_VCH_001] ...
Module	Component	Gán vào trường Components của Jira.
Rule ID	Issue Links (Tests / Covers)	Tự động liên kết Issue Key của Story/Requirement tương ứng.

Viewpoint ID	Labels	Gắn nhãn phân loại: ví dụ VP_SECURITY, VP_BOUNDARY.
Mục Đích Kiểm Thử	Summary / Description	Trở thành tóm tắt tiêu đề của Jira Test Issue.
Tiền Điều Kiện	Xray Precondition	Đưa vào khối Precondition của Xray.
Các Bước Thực Hiện	Manual Test Steps (Action & Data)	Tách từng bước (Step 1, Step 2...) thành các dòng Step riêng biệt trong Xray.
Kỳ Vọng Đầu Ra	Expected Result	Kết quả kỳ vọng tương ứng với từng Step trong Xray.

Bảng 9.1: Bảng ánh xạ trường từ Master Spec sang Jira Xray

Ánh Xạ Sang Redmine Issue Tracker

Trường Trong run_defects.md	Trường Trong Redmine	Quy cách chuyển đổi
Defect ID	Subject Prefix	[BUG-VCH-01] Sai lệch số tiền giảm giá...
Severity	Priority	Critical → Urgent; High → High; Medium → Normal.
Failed Test Case ID	Custom Field: Test Case	Lưu mã Test Case gây lỗi để truy xuất nguồn gốc.
Steps to Reproduce	Description	Tự động định dạng bằng Markdown các bước tái hiện lỗi.
Evidence Path	Attachments	Đính kèm các ảnh chụp màn hình từ thư mục evidence/.

Bảng 9.2: Bảng ánh xạ trường từ Báo Cáo Lỗi sang Redmine Defect

9.3. Cơ Chế Xuất Tập Chuẩn UTF-8 Có BOM

Khi xuất dữ liệu ra tập CSV để người dùng nhập thủ công qua giao diện web của Jira hoặc Redmine, một lỗi kỹ thuật cực kỳ phổ biến trong môi trường Windows là hiện tượng vỡ font chữ tiếng Việt (ký tự bị biến thành ký tự lạ như Nguyá»...). Hệ thống xử lý triệt để vấn đề này bằng cách tự động chèn Byte Order Mark (BOM: 0xEF, 0xBB, 0xBF) vào đầu tập tin CSV:

```
[Phương thức ghi tập CSV UTF-8 có BOM trong Node.js (javascript)]
// Chèn UTF-8 BOM để Excel và Jira nhận diện chính xác font tiếng Việt
const BOM = '\uFEFF';
const csvContentWithBom = BOM + generatedCsvData;
fs.writeFileSync(outputPath, csvContentWithBom, 'utf8');
```

CHƯƠNG 10

QUẢN TRỊ & KIỂM TRA AGENT (AGENT DOCTOR)

10.1. Nhu Cầu Quản Trị Hệ Thống Đa Chuyên Gia

Trong một hệ sinh thái kiểm thử Multi-Agent phức tạp với 8 Agent chuyên biệt và hàng chục công cụ, kỹ năng (Skills) nội bộ, các rủi ro kỹ thuật sau đây có thể xảy ra:

- **1. Lệnh pha cấu hình (Configuration Drift):** Một Agent bị sửa đổi cú pháp YAML Frontmatter của SKILL.md không đúng chuẩn, dẫn đến việc LLM không nạp được kỹ năng.
- **2. Thiếu hụt thư viện (Dependency Breakage):** Thiếu các gói Node.js cần thiết (mammoth, xlsx, pdf-parse) khiến Cổng Intake gặp lỗi chuyển đổi tài liệu.
- **3. Mất đồng bộ bản đồ (Out-of-Sync Map):** Tập _system_map.json không khớp với cấu trúc thư mục thực tế trong INPUT/ và OUTPUT/.
- **4. Hiệu ứng domino khi thay đổi (Blast Radius):** Một tài liệu hoặc quy tắc ở chặng đầu thay đổi nhưng các chặng sau không được cập nhật tương ứng, dẫn đến kiểm thử sai lệch.

10.2. Công Cụ Khám Sức Khỏe Tự Động (npm run agent:check)

Script agents/tools/check-agents.js thực hiện quét toàn diện qua 5 lớp kiểm tra:

- **Lớp 1: Cấu trúc thư mục lõi.** Kiểm tra sự tồn tại của 4 phân vùng bắt buộc: INPUT/, OUTPUT/, knowledge/, agents/.
- **Lớp 2: Kiểm định kỹ năng chuyên gia.** Quét toàn bộ 8 Agent trong agents/, xác nhận mỗi kỹ năng đều có tệp SKILL.md hợp lệ, kiểm tra cú pháp YAML frontmatter.
- **Lớp 3: Kiểm định tính toàn vẹn công cụ.** Kiểm tra 12 script tiện ích trong agents/tools/, đảm bảo không có lỗi cú pháp JavaScript và đầu ra chuẩn.
- **Lớp 4: Thư viện phụ thuộc.** Xác nhận các gói npm cần thiết đã được cài đặt đầy đủ trong node_modules/.
- **Lớp 5: Bản đồ tri thức.** Kiểm tra tính nhất quán giữa _system_map.json và các tệp trong knowledge/features/.

[Kết quả chẩn đoán của Agent Doctor (text)]

=== AGENT SYSTEM HEALTH CHECK ===

[OK] Core Directories: INPUT, OUTPUT, knowledge, agents verified.

[OK] QA Agents (8/8): All agents registered and valid.

[OK] Agent Skills (18/18): All SKILL.md files parsed successfully.

[OK] Internal Tools (12/12): All JS utilities verified syntax clean.

[OK] System Map: knowledge/_system_map.json is synchronized.

[OK] Dependencies: mammoth, xlsx, pdf-parse, js-yaml present.

System Health Status: 100% HEALTHY - Ready for production testing.

10.3. Phân Tích Bán Kính Tác Động (Blast Radius Analysis)

⚡ CHUỖI TÁC ĐỘNG LAN TỎA CỦA BLAST RADIUS

[NGUỒN THAY ĐỔI] Tài Liệu Hoặc Quy Tắc Mới: BA sửa SRS trong 02_ba/srs.md hoặc cập nhật rule trong features/<slug>.md.



[TẦNG 1] Tác Động Tầng Phân Tích (01 -> 04): Kích hoạt định nghĩa lại rủi ro (01), quét lại O6W (02), chọn lại viewpoints (03), tái cấu trúc test ideas (04).

↓
[TẦNG 2] Tác Động Tầng Đặc Tả & Bàn Giao: Tái sinh các Test Cases bị đổi trong 05_test_case_spec.md, tính toán lại 06_coverage_review.md, cập nhật biên trong 10_dataset.md.

↓
[TẦNG 3] Tác Động Tầng Thực Thi (runs/): Lọc lại danh sách cases cần chạy trong run_plan.md và thực thi kiểm thử hồi quy trên môi trường.

Hình 10.1: Sơ đồ trực quan đồ thị bán kính tác động khi có sự thay đổi từ tài liệu gốc

Nguyên tắc cốt lõi: Nguyên tắc xử lý khi có tác động lan tỏa:

- **Cảnh báo chủ động:** QA Leader không tự ý xóa bỏ các kết quả cũ mà lập tức đưa ra bảng cảnh báo cho người dùng, chỉ rõ danh sách các Test Cases bị ảnh hưởng trực tiếp.
- **Hiệu chỉnh khoanh vùng (Targeted Regeneration):** Thay vì phải chạy lại toàn bộ từ Chặng 1 đến Chặng 6, hệ thống chỉ kích hoạt tái tạo các phần bị tác động, bảo toàn các kịch bản kiểm thử không liên quan.

10.4. Bảng Kiểm Kế Bảo Trì Định Kỳ (Maintenance Checklist)

Tần suất	Hành động	Mục đích & Phương thức
Hàng ngày	Đồng bộ Bản đồ Hệ thống	Kích hoạt npm run map:sync sau mỗi phiên hoàn tất tác vụ để cập nhật trạng thái tính năng mới.
Hàng tuần	Chạy Agent Doctor	Kích hoạt npm run agent:check để phát hiện sớm các tệp cấu hình hoặc công cụ bị hỏng.
Khi có CR	Đánh giá Blast Radius	So sánh tài liệu mới với phiên bản cũ và rà soát lại các kịch bản kiểm thử bị tác động.
Sau mỗi Release	Lưu trữ Tầng Thực Thi	Nén và di chuyển các phiên runs/ cũ vào kho lưu trữ lịch sử để giữ không gian làm việc gọn gàng.

Bảng 10.1: Danh mục kiểm tra bảo trì định kỳ cho hệ thống

CHƯƠNG 11

HƯỚNG DẪN HỘI THOẠI & KỊCH BẢN GỖ CHAT THỰC TẾ

11.1. Lời Mở Đầu: Chuyển Giao Từ Lý Thuyết Đến Bàn Làm Việc Thực Tế

Mười chương trước trong tài liệu này đã cung cấp một bức tranh toàn diện và sâu sắc về mặt kiến trúc: từ 4 trụ cột triết lý kỹ thuật, cấu trúc 4 phân vùng làm việc, quy trình gác cổng số 0 (Gate 0), Bộ não tri thức vĩnh viễn (SSOT), mô hình điều phối Một cửa của QA Leader, cho đến quy trình thiết kế kiểm thử 6 chặng chuẩn mực, kỹ thuật Blueprint JSON, tích hợp Jira/Redmine và công cụ Bác sĩ Agent Doctor. Toàn bộ những nội dung đó giải thích bản chất 'Tại sao' (Why) và 'Cơ chế hoạt động ngầm' (Under the Hood) của hệ sinh thái.

Chuyển giao hành động: Chương 11 này được thiết kế để trở thành Sổ Tay Thực Chiến (Hands-On Playbook) dành riêng cho người dùng hàng ngày – bao gồm Tester, QA Lead, Business Analyst (BA), Product Owner (PO) và Developer. Khi ngồi trước bàn làm việc thực tế, bạn không cần phải ghi nhớ những cấu trúc phức tạp hay cú pháp kỹ thuật nặng nề. Chương này trả lời trực tiếp cho bạn 3 câu hỏi cốt tử:

- **1. Gõ câu gì trên chat:** Mở khung chat IDE lên thì gõ câu thoại tiếng Việt nào để ra việc chuẩn xác nhất?
- **2. Kịch bản từng bước:** Mỗi kịch bản thực tế (dự án mới, BA chốt rule, chạy test ticket, sinh data) được xử lý từng bước ra sao?
- **3. Hành động kỹ thuật tương ứng:** Hệ thống và Agent đã tự động thực thi những lệnh script ngầm nào ở hậu trường để trả về kết quả cho bạn?

NGUYÊN TẮC VẬN HÀNH BẮT BIẾN TẠI BÀN LÀM VIỆC

ĐIỀU KHOẢN THỰC THI THỰC CHIẾN:

- Mọi tương tác hàng ngày của người dùng đều diễn ra 100% bằng tiếng Việt tự nhiên trong khung chat.
- Bạn KHÔNG CẦN và KHÔNG PHẢI gõ các lệnh dòng lệnh phức tạp (Zero-CLI).
- Người dùng CHỈ CẦN giao tiếp với QA Leader (Một cửa duy nhất), không cần nhớ tên từng chuyên gia con.
- Các lệnh CLI trong chương này được liệt kê song song nhằm giúp Power User hoặc Kỹ sư quản trị nắm rõ cơ chế ngầm.

11.2. Bảng Tra Cứu 'Cheat Sheet' Câu Lệnh Gõ Chat Thông Dụng Nhất

Dưới đây là bảng tra cứu nhanh (Quick Prompt Cheat Sheet) ánh xạ giữa Mục đích công việc thực tế, câu gõ mẫu trên khung chat, lệnh script ngầm mà Agent kích hoạt ở hậu trường và sản phẩm đầu ra kỳ vọng:

Mục Đích Công Việc	Câu Gõ Mẫu Trên Khung Chat	Lệnh Ngầm / Công Cụ	Sản Phẩm Đầu Ra Kỳ Vọng
Nắm tiến độ & làm tiếp	'Tiến độ thế nào rồi?' hoặc 'Hôm nay làm gì tiếp?'	Đọc OUTPUT/**/OO_plan.md	Bảng Dashboard tiến độ tổng và đề xuất bước đi tiếp theo (Zero-Path).
Nạp tài liệu mới (Gate 0)	'Tôi vừa thêm tài liệu SRS vào thư mục INPUT, xử lý giúp tôi'	npm run intake	Báo cáo Gap Assessment, tự đổi .docx -> .md, cập nhật _system_map.json.

Mục Đích Công Việc	Câu Gõ Mẫu Trên Khung Chat	Lệnh Ngâm / Công Cụ	Sản Phẩm Đầu Ra Kỳ Vọng
Khởi tạo tính năng mới	'Khởi tạo tính năng mới có tên order-management'	npm run knowledge:new order-management	Tạo tệp tri thức SSOT mới: knowledge/features/order-management.md.
Phân tích Chặng 1 & 2	'Bắt đầu phân tích tính năng voucher-stacking từ Chặng 1 đến 2'	Agent: qa-analyst	Sinh 00_plan.md, 01_requirement_risk_summary.md và 02_missing_rule_report.md.
Cung cấp phản hồi từ BA	'BA vừa chốt rồi nhé: [nội dung quy tắc chốt]. Cập nhật và làm tiếp'	Agent: qa-lead	Ghi nhận SSOT, đóng câu hỏi Q-xx, chuyển Cổng 2 sang PASS, mở Chặng 3.
Thiết kế Viewpoint & Idea	'Tiếp tục thực hiện Chặng 3 và Chặng 4 cho tính năng này'	Agent: qa-analyst	Sinh 03_viewpoint_report.md và 04_test_idea_report.md (Bảng Giữ/Bỏ).
Lập Blueprint Test Cases	'Lập blueprint JSON cho toàn bộ test case của voucher-stacking'	Agent: qa-test-design	Sinh 05_test_blueprint.json (bản thiết kế khung nhẹ, bao phủ 100% phạm vi).
Sinh chi tiết từng Lô	'Sinh chi tiết cho Lô 1 (TC-001 đến TC-050)'	Agent: qa-test-design	Sinh testcases/batch_01.md chuẩn 8 trường không bị tràn token.
Gộp các Lô Test Case	'Gộp tất cả các batch test case lại thành file spec tổng'	npm run testcases:merge <slug>	Hợp nhất tự động thành Master Spec hoàn chỉnh: 05_test_case_spec.md.
Rà soát độ phủ 3 góc nhìn	'Rà soát độ phủ 3 góc nhìn và nghiệm thu cho tính năng này'	Agent: qa-test-design	Sinh 06_coverage_review.md với bảng chấm điểm Rule, Viewpoint, Risk.
Sinh 50 bộ dữ liệu test	'Sinh 50 bộ dữ liệu test khách hàng kèm số điện thoại Viettel và voucher'	npm run data:gen -- --count 50	Sinh 10_dataset.md hoàn tất trong 0.05s, tiêu tốn 0 token LLM.
Chạy test cho Ticket cụ thể	'Chạy test Ticket BUG-892 trên môi trường: https://staging.myshop.vn'	Agent: qa-automation	Khởi tạo runs/RUN-01_BUG-892/ (run_plan.md, run_result.md, run_defects.md).
Khám sức khỏe hệ thống	'Khám sức khỏe toàn bộ hệ thống agent giúp tôi'	npm run agent:check	Báo cáo chẩn đoán Agent Doctor 5 lớp kiểm tra toàn diện.
Đồng bộ bản đồ hệ thống	'Đồng bộ lại bản đồ hệ thống sau khi làm xong'	npm run map:sync	Đồng bộ hóa các bảng định tuyến trong

Mục Đích Công Việc	Câu Gõ Mẫu Trên Khung Chat	Lệnh Ngầm / Công Cụ	Sản Phẩm Đầu Ra Kỳ Vọng
			knowledge/_system_map.json.

Bảng 11.1: Bảng tra cứu Quick Prompt Cheat Sheet ánh xạ câu chat và hành động kỹ thuật

11.3. Kịch Bản 1: Khởi Tạo Dự Án Mới & Cổng Tiếp Nhận Số 0 (Gate 0)

Khi bạn bắt đầu một tính năng mới hoặc nhận bàn giao một gói tài liệu từ khách hàng, BA hoặc Tech Lead, bước đầu tiên là đưa tài liệu vào hệ thống để chuẩn hóa định dạng và đánh giá khoảng trống thông tin.

⚡ QUY TRÌNH TIẾP NHẬN TÀI LIỆU SỐ 0 THỰC TẾ

[BƯỚC 1] Người dùng thả tệp vào INPUT/: Thả srs_voucher.docx vào thư mục INPUT/voucher-stacking/02_ba/.

↓
[BƯỚC 2] Gõ lệnh trên khung chat: 'Tôi vừa thêm tài liệu SRS vào INPUT/voucher-stacking, xử lý giúp tôi.'

↓
[BƯỚC 3] QA Leader chạy Gate 0: Kích hoạt npm run intake chuyển đổi .docx sang .md, thực hiện Gap Assessment.

↓
[BƯỚC 4] Báo cáo căn chỉnh mục tiêu: QA Leader đưa bảng đánh giá rủi ro thiếu tài liệu và đề xuất Mode kiểm thử phù hợp.

Hình 11.1: Các bước thực chiến tiếp nhận tài liệu và căn chỉnh mục tiêu

🔴 Hội Thoại Thực Tế: Tiếp Nhận & Khởi Tạo Dự Án Mới

👤 Tester / Người dùng:

'Chào QA Leader! Tôi vừa nhận file đặc tả nghiệp vụ srs_voucher_v1.docx từ BA và đã thả vào thư mục INPUT/voucher-stacking/02_ba/. Hãy kiểm tra và xử lý giúp tôi.'

👤 QA Leader:

'Chào bạn! Tôi đã tiếp nhận tài liệu và kích hoạt Cổng Tiếp Nhận Số 0 (Intake Gate 0):'

1. Chuyển đổi định dạng: Đã tự động chuyển đổi srs_voucher_v1.docx → srs_voucher_v1.md sạch sẽ, loại bỏ rác XML.

2. Đánh giá khoảng trống (Gap Assessment):

- [ĐẦY ĐỦ] Ngăn 02_ba: Đã có tài liệu SRS quy định các điều kiện áp dụng mã voucher.
- [THIẾU] Ngăn 03_dev: Chưa có tài liệu API Swagger hoặc DB Schema. (Giả định: Xử lý theo Black-box Testing).
- [THIẾU] Ngăn 04_design: Chưa có link Figma giao diện giỏ hàng. (Giả định: Giao diện web responsive tiêu chuẩn).

3. Căn chỉnh mục tiêu đầu ra (Outcome Alignment):

Tôi đề xuất thực hiện Mode 2: Thiết Kế Test Cases Chi Tiết + Bộ Dữ Liệu Kiểm Thử (Manual + Test Data).

Bạn có đồng ý với phương án trên để tôi bắt đầu Chặng 1: Phân tích Rủi ro không?'

[Lệnh ngầm tương ứng của Kịch bản 1 (bash)]

```
# Lệnh CLI tương đương chạy ngầm ở hậu trường:
node agents/tools/intake.js
# Hoặc npm script:
npm run intake
```

11.4. Kịch Bản 2: Bóc Tách Kế Hở 06W & Quy Trình Tháo Gỡ Rào Cản Với BA

Trong quá trình kiểm thử phần mềm thực tế, lỗ hổng lớn nhất thường nằm ở các điểm tài liệu **KHÔNG ĐỀ CẬP** (Unstated Requirements). Khi phát hiện kế hở logic, Agent bắt buộc phải dừng lại ở Cổng Chặng 2 với kết luận ASK chứ không được tự suy diễn.

Bước 1: QA Leader Phát Hiện Kế Hở & Dừng Lại Tại Cổng ASK

 Tester / Người dùng:

'Bắt đầu phân tích tính năng voucher-stacking từ Chặng 1 đến Chặng 2 nhé.'

 QA Leader:

'Tôi đã giao việc cho chuyên gia QA Analyst để thực thi Chặng 1 và Chặng 2:

- Đã hoàn thành 01_requirement_risk_summary.md: Trích xuất 8 quy tắc kinh doanh đã xác nhận (BR-01 đến BR-08).
- Đã hoàn thành 02_missing_rule_report.md: Quét lưới 06W phát hiện 2 kế hở nghiêm trọng:
 1. [MR-01 - W4 Where]: Tài liệu chưa nêu rõ voucher giảm giá phí vận chuyển có được áp dụng đồng thời với voucher giảm giá sản phẩm không?
 2. [MR-02 - W6 What If]: Khi khách hàng áp dụng cùng lúc 2 voucher % giảm giá, hệ thống sẽ cộng dồn (20% + 10% = 30%) hay chỉ lấy voucher cao nhất?

CỔNG CHẶNG 2: KẾT LUẬN = ASK (TẠM DỪNG)

Quy tắc bất biến: Hệ thống không tự suy đoán nghiệp vụ tài chính. Bạn hãy gửi 2 câu hỏi trên cho BA để làm rõ trước khi chúng ta thiết kế Test Cases!

Hành động của Tester: Sau khi nhận được câu hỏi từ Agent, Tester chỉ cần sao chép nội dung câu hỏi mở gửi sang kênh trao đổi của dự án (Slack, Microsoft Teams, hoặc Jira Comment). Khi BA phản hồi, Tester dán câu trả lời vào chat:

Bước 2: Cung Cấp Phản Hồi Từ BA & Mở Khóa Tiến Độ Tự Động

 Tester / Người dùng:

'BA vừa chốt câu hỏi rồi nhé:

1. Voucher vận chuyển được cộng dồn độc lập với voucher giảm giá hàng hóa.
 2. Với 2 voucher giảm giá % cùng loại, hệ thống chỉ lấy voucher có giá trị giảm cao nhất, **TUYỆT ĐỐI KHÔNG CỘNG DỒN**.
- Hãy cập nhật vào tri thức và làm tiếp cho tôi.'

 QA Leader:

'Rất xuất sắc! Rào cản nghiệp vụ đã được tháo gỡ hoàn toàn. Tôi đã thực hiện các hành động sau:

1. Cập nhật SSOT: Bổ sung 2 quy tắc mới BR-09 và BR-10 vào knowledge/features/voucher-stacking.md.
2. Đóng câu hỏi: Chuyển trạng thái MR-01 và MR-02 từ Open → Confirmed.
3. Mở khóa Cổng Chặng 2: Chuyển Verdict từ ASK → PASS trong 00_plan.md.

Tôi đã kích hoạt chuyên gia QA Analyst để tự động chuyển tiếp sang Chặng 3 (Lựa chọn Viewpoints) ngay bây giờ!

11.5. Kịch Bản 3: Thiết Kế Test Cases Quy Mô Lớn (Blueprint, Batching & Assembly)

Khi một tính năng có quy mô lớn dự tính sinh ra từ 80 đến hàng trăm test cases, nếu yêu cầu AI sinh toàn bộ trong một lần sẽ dẫn đến hiện tượng tràn context window, cắt cụt nội dung hoặc suy giảm chất lượng kiểm thử. Dưới đây là kịch bản gõ chat chuẩn 3 giai đoạn:

⚡ QUY TRÌNH THIẾT KẾ TEST CASE QUY MÔ LỚN THỰC TẾ

[GIAI ĐOẠN 1] Sinh Blueprint Khung (~30 token/case): Gõ chat yêu cầu tạo 05_test_blueprint.json bao quát 100% mục tiêu.



[GIAI ĐOẠN 2] Sinh Chi Tiết Từng Lô (Chunked Batching): Gõ chat sinh Batch 1 (50 cases), Batch 2 (50 cases)... vào thư mục testcases/.



[GIAI ĐOẠN 3] Gộp Tổng Thể (Assembly Engine): Gõ chat 'Gộp test case' → Hệ thống tự chạy npm run testcases:merge <slug>.

Hình 11.2: Quy trình 3 bước xử lý hàng trăm test case chống tràn token



Hội Thoại Thực Tế: Thiết Kế Test Cases Bằng Blueprint & Batching

 Tester / Người dùng: 'Lập blueprint JSON cho toàn bộ test case của voucher-stacking nhé.'

 QA Leader: 'Đã hoàn thành! 05_test_blueprint.json đã ghi nhận 95 test case phân bổ qua 6 Viewpoints. Bây giờ tôi đề xuất chia thành 2 lô: Lô 1 (TC-001 -> TC-050) và Lô 2 (TC-051 -> TC-095). Bạn muốn sinh Lô nào trước?'

 Tester / Người dùng: 'Sinh chi tiết Lô 1 đi.'

 QA Leader: 'Chuyên gia QA Test Design đã hoàn thành sinh chi tiết 50 Test Cases chuẩn 8 trường vào OUTPUT/voucher-stacking/testcases/batch_01.md. Mời bạn kiểm tra hoặc yêu cầu sinh tiếp Lô 2.'

 Tester / Người dùng: 'Sinh tiếp Lô 2 rồi gộp toàn bộ lại cho tôi.'

 QA Leader: 'Đã sinh xong batch_02.md (45 cases) và tự động kích hoạt công cụ gộp: Đã tạo thành công Master Spec hoàn chỉnh gồm 95 Test Cases tại OUTPUT/voucher-stacking/05_test_case_spec.md!'

[Lệnh gộp Master Spec ở hậu trường (bash)]

```
# Lệnh CLI nội bộ tự động gộp các batch (Agent tự chạy ngầm):
node agents/tools/merge-testcases.js voucher-stacking
# Hoặc npm script:
npm run testcases:merge voucher-stacking
```

11.6. Kịch Bản 4: Động Cơ Sinh Dữ Liệu Kiểm Thử Tự Động (0-Token LLM Engine)

Khi cần dữ liệu kiểm thử thực tế (số điện thoại Việt Nam hợp lệ theo nhà mạng, họ tên chuẩn người Việt, giá trị tiền tệ có dấu phân cách hàng nghìn, giá trị biên), bạn không cần bảo AI gõ từng dòng dữ liệu vừa chậm vừa tốn token. Bạn chỉ cần yêu cầu số lượng:



Hội Thoại Thực Tế: Kích Hoạt Sinh Dữ Liệu Tốc Độ Cao

 Tester / Người dùng:

'Tôi cần 50 bộ dữ liệu test giỏ hàng: gồm họ tên khách hàng Việt Nam, số điện thoại Viettel, số tiền đơn hàng từ 150.000 đ đến 5.000.000 đ, mã voucher bắt đầu bằng SALES, và phương thức thanh toán Momo hoặc VNPay.'

 QA Leader:

'Tôi đã định nghĩa schema cấu hình siêu nhẹ và kích hoạt Động cơ sinh dữ liệu nội bộ (Data Generator Engine):

- ⚡ Tốc độ sinh: 0.04 giây cho 50 bản ghi.
- ⚡ Chi phí token LLM: 0 token.
- ⚡ Vị trí tệp: OUTPUT/voucher-stacking/10_dataset.md (kèm cả bảng Markdown trực quan và tệp CSV có UTF-8 BOM).

Bạn có thể mở tệp trực tiếp trong IDE hoặc mở trên Excel để xem toàn bộ dữ liệu mẫu!

[Lệnh sinh dữ liệu tốc độ cao (0 Token LLM) (bash)]

Lệnh CLI tương đương ở hậu trường:

```
npm run data:gen -- --slug voucher-stacking --schema dataset_schema.json --count 50
```

STT	Họ Và Tên	Số Điện Thoại	Giá Trị Đơn Hàng	Mã Voucher	Phương Thức
01	Nguyễn Hoàng Nam	0983.412.569 (Viettel)	450.000 đ	SALES-8X2M9K	MOMO
02	Trần Thị Minh Phương	0971.882.341 (Viettel)	1.250.000 đ	SALES-3P9L7A	VNPAY
03	Lê Văn Hùng	0968.125.790 (Viettel)	150.000 đ (Giá trị biên min)	SALES-5T1V4C	MOMO
04	Phạm Quốc Tuấn	0989.554.218 (Viettel)	5.000.000 đ (Giá trị biên max)	SALES-9Q8D2Z	VNPAY
...	... (Đủ 50 bộ dữ liệu)	...	...	...	...

Bảng 11.2: Trích dẫn 4/50 bộ dữ liệu kiểm thử được sinh tự động trong 10_dataset.md

11.7. Kịch Bản 5: Quản Lý Phiên Thực Thi Độc Lập & Báo Cáo Defect (Tầng runs/)

Khi kiểm thử cho một Ticket cụ thể (ví dụ: BUG-892 hoặc CR-105), bạn không làm bản Master Spec mà yêu cầu QA Leader khởi tạo một phiên thực thi độc lập (Session) trong thư mục runs/:

 **Hội Thoại Thực Tế: Thực Thi Kiểm Thử & Lập Báo Cáo Lỗi**

 Tester / Người dùng:

'Chạy test cho Ticket BUG-892 (Sửa lỗi không áp dụng được voucher trên App mobile) trên môi trường Staging: <https://staging.myshop.vn> nhé.'

 QA Leader:

'Đã rồi! Tôi đã thiết lập phiên thực thi độc lập tại:

📁 OUTPUT/voucher-stacking/runs/RUN-01-BUG-892/

1. Lọc kịch bản kiểm thử (run_plan.md): Trích xuất 12 Test Cases trọng tâm liên quan đến API tính toán voucher giỏ hàng từ Master Spec.

2. Kết quả thực thi (run_result.md): 10 Passed, 2 Failed.
3. Bằng chứng kiểm thử (evidence/): Đã chụp 2 ảnh màn hình lỗi toast đỏ hiển thị sai số tiền.
4. Hồ sơ lỗi (run_defects.md): Đã tự động khởi tạo Defect DEF-BUG892-01 (Mức độ: High - Chiết khấu voucher bị tính âm).

Bạn có muốn tôi hỗ trợ đồng bộ Defect này lên Jira Issue Tracker ngay không?

11.8. Kịch Bản 6: Vận Hành Bảo Trì & Bác Sĩ Hệ Thống (Agent Doctor)

Sau mỗi chu kỳ chạy hoặc vào đầu mỗi tuần làm việc, Tester hoặc Tech Lead có thể yêu cầu kiểm tra sức khỏe toàn bộ hệ thống để đảm bảo mọi chuyên gia và công cụ đều sẵn sàng:

✓ **Hội Thoại Thực Tế: Khám Sức Khỏe Toàn Diện Hệ Thống**
Tester / Người dùng: 'Khám sức khỏe toàn bộ hệ thống agent giúp tôi.'

🗣️ QA Leader:

'Tôi đã kích hoạt công cụ Bác sĩ Agent Doctor quét qua 5 lớp kiểm tra toàn diện:

- Phân vùng lỗi: Đầy đủ 4 thư mục INPUT, OUTPUT, knowledge, agents. (Trạng thái: OK)
- Chuyên gia QA: Toàn bộ 8 Agent đã đăng ký và cấu hình hợp lệ. (Trạng thái: OK)
- Kỹ năng (Skills): 18/18 tệp SKILL.md có cú pháp YAML frontmatter chuẩn xác. (Trạng thái: OK)
- Công cụ nội bộ: 12 script tiện ích trong agents/tools/ cú pháp hợp lệ. (Trạng thái: OK)
- Bản đồ hệ thống: knowledge/_system_map.json hoàn toàn đồng bộ với các thư mục. (Trạng thái: OK)

🎉 **KẾT LUẬN:** Hệ thống đạt trạng thái 100% HEALTHY - Sẵn sàng cho mọi tác vụ kiểm thử!

[Lệnh bảo trì hệ thống định kỳ ở hậu trường (bash)]

Lệnh CLI kiểm tra sức khỏe hệ thống (Agent Doctor):
npm run agent:check

Lệnh CLI đồng bộ bản đồ vệ tinh toàn hệ thống:
npm run map:sync

11.9. Năm Nguyên Tắc Vàng Dành Cho Người Thực Chiến

Ghi nhớ nhanh: Để đạt hiệu suất tối đa khi làm việc hàng ngày với hệ sinh thái QA Agent, người dùng hãy luôn ghi nhớ 5 nguyên tắc vàng:

- **1. Tin tưởng Bản đồ:** Trước khi bắt đầu bất kỳ phiên làm việc nào, Agent luôn đọc _system_map.json để định tuyến chuẩn xác, bạn không cần phải chỉ đường dẫn file chi tiết.
- **2. Kế hoạch chống tràn ngữ cảnh:** Mọi tác vụ đều có OO_plan.md ghi nhận lộ trình rõ ràng, giúp bạn tắt máy nghĩ trưa hoặc đổi từ Antigravity sang Claude Code mà không bị mất dấu công việc.
- **3. Trải nghiệm không gõ đường dẫn:** Khi mở khung chat, chỉ cần gõ 'Tiến độ thế nào?' hoặc 'Làm tiếp', Agent tự tìm ra task đang dở dang mà bạn không cần gõ lại đường dẫn dài dòng.
- **4. Tôn trọng Cổng Dừng:** Khi Agent báo Verdict là ASK ở Chặng 2, tuyệt đối không giục Agent đoán bừa. Hãy gửi câu hỏi cho BA để đảm bảo chuẩn FACT 100%.
- **5. Giữ sạch Master Spec (Runs Isolation):** Toàn bộ 100 test case gốc nằm nguyên vẹn trong Master Spec (05_test_case_spec.md). Khi chạy thực tế theo từng ticket, hãy dùng thư mục runs/ để giữ tài liệu thiết kế luôn sạch sẽ.